

Resumen: JavaScript Avanzado (2.^a parte)

Fastbook de Frontend · Autor original: Robert Da Corte

Este documento resume los seis módulos del fastbook, centrados en conceptos avanzados de JavaScript: la herencia basada en prototipos, la gestión de memoria y el rendimiento, y tres herramientas clave del ecosistema moderno: npm, los transpiladores (Babel) y la biblioteca reactiva RxJS.

1. Prototipos y herencia prototipal

A diferencia de lenguajes con herencia basada en clases, **JavaScript usa herencia basada en prototipos**: los objetos heredan propiedades y métodos directamente de otros objetos.

Ideas clave

- Cada objeto tiene una referencia interna **[[Prototype]]** a otro objeto (su 'prototipo'). En navegadores/Node se puede inspeccionar con `__proto__` (no recomendado en producción).
- Si una propiedad/método no existe en el objeto, JavaScript la busca en la cadena de prototipos hasta encontrarla o llegar al final (devuelve **undefined**).
- **Object** es el punto de partida: todos los objetos heredan, tarde o temprano, de **Object.prototype** (final de la cadena, seguido de null).
- Métodos heredados de **Object.prototype**: **constructor**, **hasOwnProperty()**, **isPrototypeOf()**, **toString()**, **valueOf()**.

Funciones constructoras

Una **función constructora** es una plantilla para crear objetos (por convención, con mayúscula inicial). Al invocarla con **new**, JavaScript: crea un objeto vacío, enlaza su prototipo al de la función, vincula **this** al nuevo objeto, ejecuta el código del constructor y devuelve el objeto resultante.

Class (ES6)

La palabra clave **class** es 'sugar syntax': por debajo sigue usando herencia prototipal. Incluye un método especial **constructor** que inicializa cada instancia. La herencia entre clases se logra con **extends** (clase hija) y **super** (llama al constructor de la clase padre).

Idea central: en JS, la herencia se basa en una cadena de prototipos, no en clases clásicas; 'class' es solo una forma más clara de escribir ese mismo mecanismo.

2. Memoria y optimización de rendimiento

JavaScript usa un **garbage collector** (recolector de basura) con el algoritmo **mark-and-sweep**: marca las variables en uso y libera la memoria de las que no lo están. Sin embargo, malas prácticas pueden causar **memory leaks** (fugas de memoria).

Una fuga de memoria ocurre cuando objetos ya innecesarios no se liberan porque todavía existen referencias a ellos en algún punto del código.

Causas comunes de fugas de memoria

- **Variables globales** innecesarias que nunca se liberan.
- **Event listeners** no eliminados al dejar de ser necesarios.
- **Referencias circulares** (menos problemáticas hoy, pero aún relevantes).
- **Closures** mal gestionadas que retienen referencias innecesarias.
- **Timers/Intervals** (setTimeout/setInterval) no limpiados con clearTimeout/clearInterval.

Herramientas de monitorización

- **DevTools de Chrome** (pestaña 'Memory'): permite tomar y comparar instantáneas de memoria.
- **Node.js con --inspect**: habilita depuración conectando DevTools.
- **Lighthouse**: análisis automatizado de rendimiento web.

3. Npm

Node.js impulsó el ecosistema frontend moderno a través de su gestor de paquetes: **npm**, que permite instalar, compartir y gestionar bibliotecas, herramientas y componentes reutilizables (React, Vue, Lodash, Babel, Webpack, ESLint, etc.).

Flujo básico

- **npm init** — inicializa el proyecto (asistente interactivo) o **npm init -y** para valores predeterminados; genera **package.json**.
- **package.json** — archivo central del proyecto: dependencias, scripts, metadatos.
- **npm install <paquete>** — instala una dependencia y crea la carpeta **node_modules**.

package.json define las propiedades y configuraciones del proyecto: dependencias, scripts, autor y más — es el archivo fundamental de cualquier proyecto Node/npm.

4. Transpiladores

Un **transpilador** convierte código escrito con sintaxis moderna (ES6/ES7+) a una versión compatible con entornos más antiguos (ES5), ampliando la compatibilidad entre navegadores.

Babel es el transpilador más popular del ecosistema JavaScript.

Configuración básica de Babel

- Instalar: **@babel/core** (núcleo), **@babel/cli** (línea de comandos) y **@babel/preset-env** (preset de transformación).
- Crear **.babelrc** con el preset **@babel/preset-env**.
- Agregar un script en **package.json** (ej. **babel src --out-dir dist**) y ejecutarlo con **npm run build**.

5. RxJS

RxJS es una biblioteca de programación reactiva basada en **observables**, que facilita la composición de código asíncrono o basado en eventos. Es muy usada en Angular, aunque aplicable en cualquier contexto JS.

Conceptos fundamentales

- **Observable**: colección de valores futuros/eventos; como una promesa que puede emitir múltiples valores en el tiempo.
- **Observer**: objeto con callbacks (next, error, complete) que escucha los valores emitidos.
- **Subscription**: resultado de suscribir un observer a un observable; representa la ejecución en curso.
- **Operators**: funciones puras para transformar/filtrar/combinar observables (map, filter, merge, concat...).
- **Subject**: actúa como observable y observer a la vez; permite *multicast* a múltiples observadores (a diferencia del observable regular, que es *unicast*).
- **Schedulers**: controlan cuándo se inicia la ejecución y cuándo se procesan las notificaciones (ej. `asvncScheduler`).

Es fundamental cerrar las suscripciones (**`subscription.unsubscribe()`**) cuando ya no se necesitan, para evitar memory leaks.

6. ¿Qué hemos aprendido? — Síntesis final

- El objeto **Object** es la base de la herencia y la estructura de todos los objetos en JS.
- Las **funciones constructoras y clases** permiten modelar componentes robustos y reutilizables.
- La **gestión de memoria** (evitar leaks, usar herramientas de perfilado) es clave para el rendimiento.
- **npm** da acceso al ecosistema colectivo de bibliotecas y herramientas.
- Los **transpiladores** (Babel) permiten escribir código moderno manteniendo compatibilidad.
- **RxJS** aporta una nueva perspectiva reactiva para manejar flujos de datos asíncronos.